

Interpreter Design Pattern

Intent

Given a simple "language" (a grammar), define a **class per grammar rule** and an `interpret()` operation that evaluates sentences written in that language. You represent a sentence as an **abstract syntax tree (AST)** of expression objects, then evaluate it by asking the tree to interpret itself.

Here the little language is **integer arithmetic with + and -**. A string like `"5 + 3 - 2"` is turned into a tree of expression objects and evaluated to `6`.

UML class diagram

```
<<interface>> IExpression | +interpret() : int | ^ ^ ^ | | | NumberExpression
AddExpression SubtractExpression (terminal/leaf) | left,right | | left,right | +-->
left.interpret() + right.interpret() Context.parseExpression("5 + 3 - 2") -> Subtract(
Add(Num 5, Num 3), Num 2 ) -> interpret() = 6
```

The players

```
expressions/IExpression the grammar contract: interpret() → int
expressions/terminal/NumberExpression TERMINAL rule – a literal number (a leaf)
expressions/nonterminal/AddExpression NON-TERMINAL rule – left + right
/SubtractExpression NON-TERMINAL rule – left - right context/Context parses a string into
an expression tree InterpreterDesignPattern the demo – parse "5 + 3 - 2", interpret it
```

Two kinds of grammar rules, straight out of the pattern:

- **Terminal expression** — an atom that evaluates by itself (`NumberExpression`). It's a leaf of the tree.
- **Non-terminal expression** — a rule built out of *other* expressions (`AddExpression`, `SubtractExpression`). It's an internal node with children.

The code, line by line

`IExpression` — the grammar contract

```
public interface IExpression { int interpret(); }
```

- One method: `interpret()` returns the `int` value of this expression. Every node in the tree — leaf or internal — implements it.
- This uniform interface is what lets a non-terminal treat its children abstractly: an `AddExpression` just calls `interpret()` on its left and right, never caring whether they're numbers or nested sub-expressions.

`NumberExpression` — the terminal rule

```
public class NumberExpression implements IExpression { private int number; public  
NumberExpression(int number) { this.number = number; } @Override public int interpret() {  
return number; } }
```

- Wraps a single literal integer. Its `interpret()` is the **base case** of the recursion: it just returns its own value, with no further calls.
- These are the **leaves** of the AST — the recursion bottoms out here.

`AddExpression` / `SubtractExpression` — the non-terminal rules

```
public class AddExpression implements IExpression { private IExpression leftExpression;  
private IExpression rightExpression; public AddExpression(IExpression leftExpression,  
IExpression rightExpression) { this.leftExpression = leftExpression; this.rightExpression  
= rightExpression; } @Override public int interpret() { return leftExpression.interpret()  
+ rightExpression.interpret(); } }
```

- Each holds **two child `IExpressions`** (typed as the interface, so a child can be a number or another whole sub-tree).
- **`interpret()` is recursive:** it evaluates its left child, evaluates its right child, and combines them (+ here, - in `SubtractExpression`). This is the **composite recursion** that walks the tree — an internal node's value is defined in terms of its children's values.
- `SubtractExpression` is identical except it returns `left - right`.

`Context` — turning a string into a tree (the parser)

```
public IExpression parseExpression(String expression) { String[] tokens =  
expression.split(" "); IExpression result = new  
NumberExpression(Integer.parseInt(tokens[0])); for (int i = 1; i < tokens.length - 1; i +=  
2) { String operator = tokens[i]; IExpression right = new  
NumberExpression(Integer.parseInt(tokens[i + 1])); if (operator.equals("+")) { result =  
new AddExpression(result, right); } else if (operator.equals("-")) { result = new
```

```
SubtractExpression(result, right); } } return result; }
```

- `split(" ")` breaks `"5 + 3 - 2"` into tokens `["5", "+", "3", "-", "2"]`. (This tiny "lexer" assumes single-space separation.)
- `result` starts as the first number — `NumberExpression(5)`.
- The loop walks the rest in (operator, number) pairs (`i += 2`): it reads an operator and the number after it, then wraps the running `result` as the left child and the new number as the right child. So the tree grows left-deep:
- after `+` 3: `result = Add( Number(5), Number(3) )`
- after `-` 2: `result = Subtract( Add(Number(5),Number(3)), Number(2) )`
- `i < tokens.length - 1` stops one short so `tokens[i + 1]` is always a valid operand — it never reads past the end.
- The returned `result` is the **root** of the AST. Nothing has been evaluated yet — parsing only *builds* the tree; `interpret()` evaluates it.

InterpreterDesignPattern — the demo

```
String inputExpression = "5 + 3 - 2"; Context context = new Context(); IExpression  
expression = context.parseExpression(inputExpression); // build the tree int result =  
expression.interpret(); // evaluate the tree System.out.println("Result: " + result); // 6
```

- **Two distinct phases:** first `parseExpression` builds the AST, then `interpret()` evaluates it. This separation is the essence of the pattern — represent the sentence as objects, then run one operation over that representation.
- `interpret()` on the root triggers the recursive cascade down to the leaves, producing 6.

Why the design decisions

Why represent the expression as a tree of objects at all?

Because it turns "evaluate this string" into "walk this structure," and the structure mirrors the grammar exactly. Each grammar rule is one class; evaluating is just polymorphic recursion. Once the AST exists you can also do *other* things with it later (pretty-print it, optimize it, translate it) by adding new operations — the representation is reusable, a plain `eval(string)` function is not.

Why split into terminal vs. non-terminal expressions?

That split **is** the grammar. A number is a terminal (it stands alone); an addition is a non-terminal (it's defined using other expressions). Modeling both as `IExpression` lets them nest arbitrarily — `AddExpression` doesn't know or care whether its children are literals or deep sub-trees, because they're all just `IExpressions`. That uniformity is what makes the recursion clean.

Why is `interpret()` recursive, and why does the interface make that possible?

A non-terminal's meaning depends on its parts, so evaluation is naturally recursive: to add, first evaluate the two operands (which may themselves be additions/subtractions). Typing the children as `IExpression` means each node calls `interpret()` on its children **without a type check** — the terminal case (`NumberExpression`) ends the recursion by returning a value directly. This is the same idea as the Composite pattern applied to grammar rules.

Why separate parsing (`context`) from interpreting (`interpret()`)?

Two responsibilities: **building** the tree from text vs. **evaluating** the tree. Keeping them apart means the expression classes know nothing about string syntax (they'd work if you built the tree by hand, `new AddExpression(new NumberExpression(5), ...)`), and the parser knows nothing about evaluation. You can change the input syntax without touching evaluation, and vice versa.

Why does this build the tree left-to-right (and what was wrong before)?

For `+` and `-` — which have **equal precedence and are left-associative** — left-to-right nesting is exactly correct: `5 + 3 - 2` must mean `(5 + 3) - 2`, and the loop produces precisely that left-deep tree.

Bug that was fixed. The original `context` used a **stack that popped two operands per operator** — that's a *postfix / Reverse-Polish* evaluator, e.g. for `"5 3 + 2 -"`. But the demo input is **infix** (`"5 + 3 - 2"`). On the first operator the stack had only one operand, so the second `pop()` threw `java.util.EmptyStackException` and the program crashed. The parser was rewritten to build the tree left-to-right from the infix input, so `"5 + 3 - 2"` now evaluates to 6.

Note this left-to-right approach is correct **only** because the language has just `+/−`. If you add `*` or `/`, it would ignore precedence (it'd compute `2 + 3 * 4` as 20, not 14) — you'd then need operator-precedence handling (e.g. the shunting-yard algorithm) or a recursive-descent parser.

Execution flow (the demo — `"5 + 3 - 2"`)

Phase 1 — parse (build the AST):

```
tokens = [ "5", "+", "3", "-", "2" ] result = Number(5) i=1: "+" 3 → result = Add(
Number(5), Number(3) ) i=3: "-" 2 → result = Subtract( Add(Number(5),Number(3)),
Number(2) ) returns the root: Subtract / \ Add Number(2) / \ Number(5) Number(3)
```

Phase 2 — interpret (evaluate the AST):

```
Subtract.interpret() ■■■ left = Add.interpret() ■ ■■■ left = Number(5).interpret() → 5
■ ■■■ right = Number(3).interpret() → 3 ■ ■■■ returns 5 + 3 = 8 ■■■ right =
Number(2).interpret() → 2 ■■■ returns 8 - 2 = 6
```

Console output:

```
Interpreter Design Pattern Evaluating Expression: 5 + 3 - 2 Result: 6
```

Notes / possible extensions (not changed in the code)

- **Adding a new operation is easy** — e.g. `MultiplyExpression` implements `IExpression` with `left * right`, plus a `*` branch in the parser. That's the pattern's sweet spot: one new class per new grammar rule. (Precedence, as noted, is the extra work.)
- **The parser is deliberately minimal** — it assumes well-formed, single-space-separated input and doesn't handle parentheses or validation. Interpreter is about *evaluating the tree*, not about industrial-strength parsing; real languages use a dedicated parser/lexer to build the AST and keep Interpreter for the evaluation step.
- **Plain main, no Spring** — the pattern is pure OO structure and needs no container.

Relationship to the other Behavioral patterns

- **Interpreter (this module)** — model a grammar as a class-per-rule tree and evaluate it recursively via `interpret()`.
- **Composite** (structural) — Interpreter's AST is a Composite; the terminal/non-terminal split is leaf vs. composite node.
- **Visitor** — often paired with Interpreter to add *many* operations over the AST (evaluate, print, type-check) without editing the expression classes.
- **Strategy / State** — delegate to a single swapped-in object; Interpreter instead walks a whole structure of objects.

Reach for Interpreter when you have a **small, stable grammar** you need to evaluate repeatedly (arithmetic, boolean rules, query filters, config mini-languages) and modeling each rule as a class keeps the evaluation clean and extensible. For large or fast-changing languages, a real parser generator is the better tool.